

CS206 Project Hypothesis Generators Used

Below are examples of generators used for creating tests using Hypothesis.

Please keep in mind that these are examples picked at random, and represent an approximation of the overall testing suite. As we generated test functions, generators were included automatically in the implementation.

```
# Test for 'preserves_length' property
@given(
    current=st.lists(st.floats(), min_size=1),
    target=st.lists(st.floats(), min_size=1),
    damping=st.floats(min_value=0, max_value=1)
)
```

Many arithmetic based functions centered around integer and floating point generators.

This responsibility was given to the AI model and refined through either manual corrections or more explicit prompts.

```
st.tuples(st.one_of(st.none(), st.integers(), st.text(), st.booleans()))
```

One example of manual corrections after a hallucination was replacing “st.anything()” with the following example. This generator was reserved for list operations that are data-type agnostic.

```
@composite
def inputs(draw):
    x0 = draw(st.floats(allow_nan=False, allow_infinity=False))
    y0 = draw(st.floats(allow_nan=False, allow_infinity=False))
    x1 = draw(st.floats(min_value=x0 + 1, allow_nan=False, allow_infinity=False))
    x = draw(st.floats(allow_nan=False, allow_infinity=False))
    clamp = draw(st.booleans())

    return x0, y0, x1, x, clamp
```

When dealing with syntax errors regarding generators, sometimes we opted for a predefined set of values that can be repicked at runtime. This avoided compilation errors of undefined variables.